

Creating Data Replica using diff MongoDB Servers and Sharding

By Shashank Kumar(24BAI10032)

0. Stop Normal MongoDB Service

Open CMD as Administrator:

```
net stop MongoDB
```

Verify:

```
sc query MongoDB
```

Should show:

```
STATE : 1 STOPPED
```

1. Create Data Folders

If you want a completely fresh replica set, delete old folders:

```
C:\data\db0  
C:\data\db1  
C:\data\db2
```

Then recreate:

```
mkdir C:\data  
mkdir C:\data\db0  
mkdir C:\data\db1
```

```
mkdir C:\data\db2
```

```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>net stop MongoDB
The MongoDB Server (MongoDB) service is stopping.
The MongoDB Server (MongoDB) service was stopped successfully.

C:\Windows\System32>sc query MongoDB

SERVICE_NAME: MongoDB
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE                : 1   STOPPED
        WIN32_EXIT_CODE       : 0   (0x0)
        SERVICE_EXIT_CODE   : 0   (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0

C:\Windows\System32>mkdir C:\data

C:\Windows\System32>mkdir C:\data\db0

C:\Windows\System32>mkdir C:\data\db1

C:\Windows\System32>mkdir C:\data\db2

C:\Windows\System32>
```

2. Start Three MongoDB Instances

Open **3 separate CMD windows**.

Window 1

```
"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" --replSet myRepSet --dbpath
C:\data\db0 --port 27018 --bind_ip localhost
```

Window 2

```
"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" --replSet myRepSet --dbpath
C:\data\db1 --port 27019 --bind_ip localhost
```

Window 3

```
"C:\Program Files\MongoDB\Server\8.3\bin\mongod.exe" --replSet myReplSet --dbpath  
C:\data\db2 --port 27020 --bind_ip localhost
```

Leave all three windows open.

3. Connect Compass

Use:

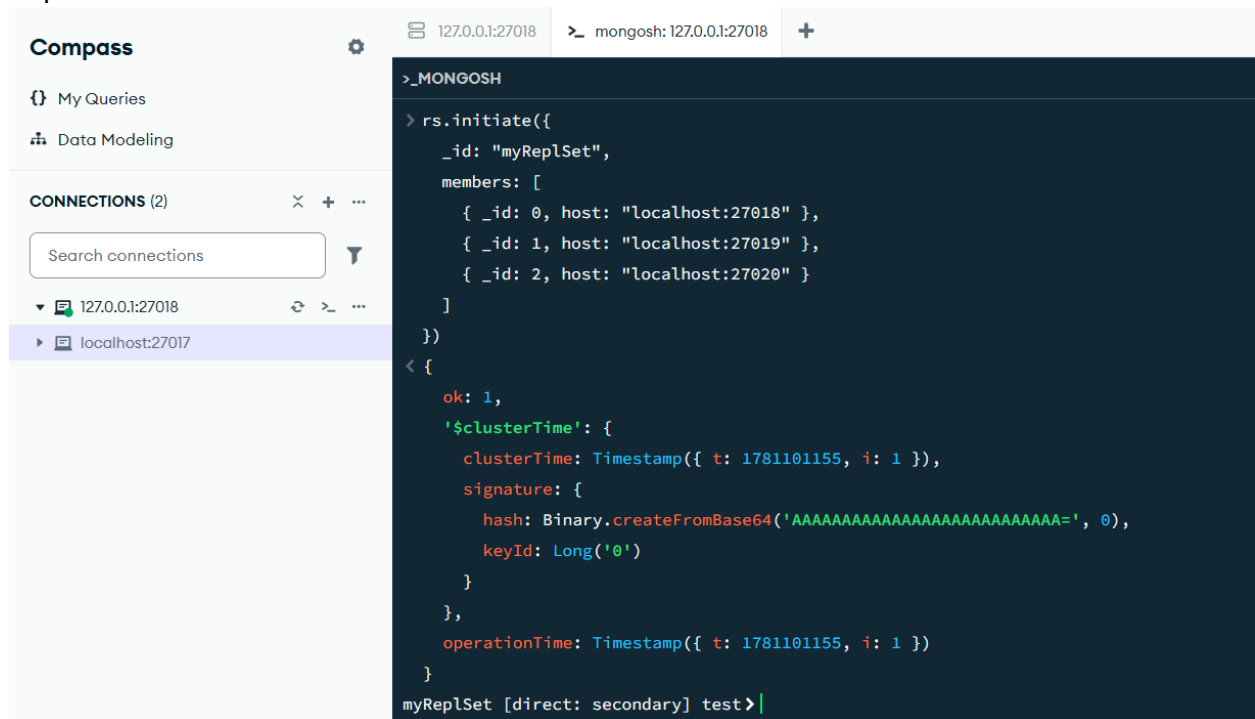
```
mongodb://127.0.0.1:27018/?directConnection=true
```

4. Open Mongosh in Compass

Run:

```
rs.initiate(  
  _id: "myReplSet",  
  members: [  
    { _id: 0, host: "localhost:27018" },  
    { _id: 1, host: "localhost:27019" },  
    { _id: 2, host: "localhost:27020" }  
  ]  
})
```

Expected:



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS (2)' panel lists two connections: '127.0.0.1:27018' and 'localhost:27017'. The main panel displays the MongoDB shell output for the command `rs.initiate({ _id: 'myRepSet', members: [{ _id: 0, host: 'localhost:27018' }, { _id: 1, host: 'localhost:27019' }, { _id: 2, host: 'localhost:27020' } ] })`. The output shows `ok: 1` and a `$clusterTime` object, indicating the replica set was successfully initialized. The prompt `myRepSet [direct: secondary] test>` is visible at the bottom of the shell window.

{ ok: 1 }

5. Verify Replica Set

Wait a few seconds.

Run:

```
rs.status()
```

Expected:

```
27017 PRIMARY
27018 SECONDARY
```

27019 SECONDARY

```
members: [
  {
    _id: 0,
    name: 'localhost:27018',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 720,
    optime: { ts: Timestamp({ t: 1781101686, i: 1 }), t: Long('1') },
    optimeDate: 2026-06-10T14:28:06.000Z,
    optimeWritten: { ts: Timestamp({ t: 1781101686, i: 1 }), t: Long('1') },
    optimeWrittenDate: 2026-06-10T14:28:06.000Z,
    lastAppliedWallTime: 2026-06-10T14:28:06.416Z,
    lastDurableWallTime: 2026-06-10T14:28:06.416Z,
    lastWrittenWallTime: 2026-06-10T14:28:06.416Z,
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1781101165, i: 1 }),
    electionDate: 2026-06-10T14:19:25.000Z,
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
```

```

    _id: 1,
    name: 'localhost:27019',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 538,
    optime: { ts: Timestamp({ t: 1781101686, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1781101686, i: 1 }), t: Long('1') },
    optimeWritten: { ts: Timestamp({ t: 1781101686, i: 1 }), t: Long('1') },
    optimeDate: 2026-06-10T14:28:06.000Z,
    optimeDurableDate: 2026-06-10T14:28:06.000Z,
    optimeWrittenDate: 2026-06-10T14:28:06.000Z,
    lastAppliedWallTime: 2026-06-10T14:28:06.416Z,
    lastDurableWallTime: 2026-06-10T14:28:06.416Z,
    lastWrittenWallTime: 2026-06-10T14:28:06.416Z,
    lastHeartbeat: 2026-06-10T14:28:13.056Z,
    lastHeartbeatRecv: 2026-06-10T14:28:12.040Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27018',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 1,
    configTerm: 1

```

(Primary could be another node as well.)

6. Test Replication

On the PRIMARY:

use college

```

db.students.insertOne({
  name: "Shashank",
  course: "MongoDB"
})

```

```
,  
> use college  
< switched to db college  
> db.students.insertOne({  
  name: "Shashank",  
  course: "MongoDB"  
})  
< {  
  acknowledged: true,  
  insertedId: ObjectId('6a2974fcd17ac32adcda6d64')  
}  
myReplSet [direct: primary] college>|
```

7. Verify on Secondary

Connect Compass to:

mongodb://127.0.0.1:27019/?directConnection=true

Open shell:

rs.secondaryOk()

use college

db.students.find()

Expected:

```
{  
  name: "Shashank",  
  course: "MongoDB"  
}
```

My Queries

Data Modeling

CONNECTIONS (3)

Search connections

127.0.0.1:27018

127.0.0.1:27019

admin

college

config

local

>_MONGOSH

> rs.secondaryOk()

< DeprecationWarning: .setSecondaryOk() is deprecated. Use .setReadPref("primaryPreferred") instead

< Setting read preference from "primary" to "primaryPreferred"

> use college

< switched to db college

> db.students.find()

< {

_id: ObjectId('6a2974fcd17ac32adcdaafd64'),

name: 'Shashank',

course: 'MongoDB'

}

myReplSet [direct: secondary] college> rs.status()

{

_id: 2,

name: 'localhost:27020',

health: 1,

state: 2,

stateStr: 'SECONDARY',

uptime: 892,

optime: { ts: Timestamp({ t: 1781102038, i: 1 }), t: Long('2') },

optimeDurable: { ts: Timestamp({ t: 1781102038, i: 1 }), t: Long('2') },

optimeWritten: { ts: Timestamp({ t: 1781102038, i: 1 }), t: Long('2') },

optimeDate: 2026-06-10T14:33:58.000Z,

optimeDurableDate: 2026-06-10T14:33:58.000Z,

optimeWrittenDate: 2026-06-10T14:33:58.000Z,

lastAppliedWallTime: 2026-06-10T14:34:08.238Z,

lastDurableWallTime: 2026-06-10T14:34:08.238Z,

lastWrittenWallTime: 2026-06-10T14:34:08.238Z,

lastHeartbeat: 2026-06-10T14:34:06.491Z,

lastHeartbeatRecv: 2026-06-10T14:34:07.222Z,

pingMs: Long('0'),

lastHeartbeatMessage: '',

syncSourceHost: 'localhost:27019',

syncSourceId: 1,

infoMessage: '',

configVersion: 1,

configTerm: 2

}

8. Test Failover

Kill the PRIMARY window (Ctrl+C).

Wait ~10 seconds.

Connect to a surviving node and run:

`rs.status()`

```
},
  members: [
    {
      _id: 0,
      name: 'localhost:27018',
      health: 0,
      state: 8,
      stateStr: '(not reachable/healthy)',
      uptime: 0,
      optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
      optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
      optimeWritten: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
      optimeDate: 1970-01-01T00:00:00.000Z,
      optimeDurableDate: 1970-01-01T00:00:00.000Z,
      optimeWrittenDate: 1970-01-01T00:00:00.000Z,
      lastAppliedWallTime: 2026-06-10T14:33:28.203Z,
      lastDurableWallTime: 2026-06-10T14:33:28.203Z,
      lastWrittenWallTime: 2026-06-10T14:33:28.203Z,
      lastHeartbeat: 2026-06-10T14:34:08.398Z,
      lastHeartbeatRecv: 2026-06-10T14:33:32.700Z,
      pingMs: Long('0'),
      lastHeartbeatMessage: 'Error connecting to localhost:27018 (127.0.0.1:27018) :: caused by :: onInvoke :: caused',
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: '',
      configVersion: 1,
      configTerm: 2
    },
  ],
}
```

Expected:

Old Primary -> health 0

One Secondary -> PRIMARY

Other Secondary -> SECONDARY

```
configTerm: 2,
},
{
  _id: 1,
  name: 'localhost:27019',
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 1058,
  optime: { ts: Timestamp({ t: 1781102048, i: 1 }), t: Long('2') },
  optimeDate: 2026-06-10T14:34:08.000Z,
  optimeWritten: { ts: Timestamp({ t: 1781102048, i: 1 }), t: Long('2') },
  optimeWrittenDate: 2026-06-10T14:34:08.000Z,
  lastAppliedWallTime: 2026-06-10T14:34:08.238Z,
  lastDurableWallTime: 2026-06-10T14:34:08.238Z,
  lastWrittenWallTime: 2026-06-10T14:34:08.238Z,
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: '',
  electionTime: Timestamp({ t: 1781101998, i: 1 }),
  electionDate: 2026-06-10T14:33:18.000Z,
  configVersion: 1,
  configTerm: 2,
  self: true,
  lastHeartbeatMessage: ''
},
{
```

```
{
  _id: 2,
  name: 'localhost:27020',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 892,
  optime: { ts: Timestamp({ t: 1781102038, i: 1 }), t: Long('2') },
  optimeDurable: { ts: Timestamp({ t: 1781102038, i: 1 }), t: Long('2') },
  optimeWritten: { ts: Timestamp({ t: 1781102038, i: 1 }), t: Long('2') },
  optimeDate: 2026-06-10T14:33:58.000Z,
  optimeDurableDate: 2026-06-10T14:33:58.000Z,
  optimeWrittenDate: 2026-06-10T14:33:58.000Z,
  lastAppliedWallTime: 2026-06-10T14:34:08.238Z,
  lastDurableWallTime: 2026-06-10T14:34:08.238Z,
  lastWrittenWallTime: 2026-06-10T14:34:08.238Z,
  lastHeartbeat: 2026-06-10T14:34:06.491Z,
  lastHeartbeatRecv: 2026-06-10T14:34:07.222Z,
  pingMs: Long('0'),
  lastHeartbeatMessage: '',
  syncSourceHost: 'localhost:27019',
  syncSourceId: 1,
  infoMessage: '',
  configVersion: 1,
  configTerm: 2
}
```

9. Recovery Test

Restart the old Primary with the same command.

Wait 15–20 seconds.

Run:

`rs.status()`

```
{
  _id: 1,
  name: 'localhost:27019',
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 1359,
  optime: { ts: Timestamp({ t: 1781102348, i: 1 }), t: Long('2') },
  optimeDate: 2026-06-10T14:39:08.000Z,
  optimeWritten: { ts: Timestamp({ t: 1781102348, i: 1 }), t: Long('2') },
  optimeWrittenDate: 2026-06-10T14:39:08.000Z,
  lastAppliedWallTime: 2026-06-10T14:39:08.574Z,
  lastDurableWallTime: 2026-06-10T14:39:08.574Z,
  lastWrittenWallTime: 2026-06-10T14:39:08.574Z,
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: '',
  electionTime: Timestamp({ t: 1781101998, i: 1 }),
  electionDate: 2026-06-10T14:33:18.000Z,
  configVersion: 1,
  configTerm: 2,
  self: true,
  lastHeartbeatMessage: ''
}
```

Expected:

```
members: [
  {
    _id: 0,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 31,
    optime: { ts: Timestamp({ t: 1781102338, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1781102338, i: 1 }), t: Long('2') },
    optimeWritten: { ts: Timestamp({ t: 1781102338, i: 1 }), t: Long('2') },
    optimeDate: 2026-06-10T14:38:58.000Z,
    optimeDurableDate: 2026-06-10T14:38:58.000Z,
    optimeWrittenDate: 2026-06-10T14:38:58.000Z,
    lastAppliedWallTime: 2026-06-10T14:39:08.574Z,
    lastDurableWallTime: 2026-06-10T14:39:08.574Z,
    lastWrittenWallTime: 2026-06-10T14:39:08.574Z,
    lastHeartbeat: 2026-06-10T14:39:07.915Z,
    lastHeartbeatRecv: 2026-06-10T14:39:08.807Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27019',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 2
  }
]
```

Old Primary -> SECONDARY

Current Primary -> remains PRIMARY

10. Clean Shutdown

In each mongod window:

Ctrl + C

After all three stop:

net start MongoDB

to return to your normal MongoDB setup.

This is the complete replica set practical from start to finish. If this is for a lab exam/viva, I'd also memorize **why 3 nodes are needed**, **what oplog is**, **what PRIMARY/SECONDARY mean**, and **what happens during failover**. Those are the most commonly asked follow-up questions.